

New PinFinder Classic Spec

Release version 1.3.8

- 1) User goes to *pinfinder.intel.com* web page
- 2) If you select “iTUFF Prober”, it will go to the specific SITE that is selected
- 3) User is able to select another Site in 2nd drop down menu
- 4) User types in a Lot
 - a. [site_ituffs] table is queried for that site to get all of the search paths for that site
 - b. Paths are searched and Operation Pull down is populated
- 5) User selects an Operation
 - a. [site_ituffs] table is queried for that site to get all of the search paths for that site
 - b. Summary file list Pull down is populated
- 6) User select a Summary (MIDAS file is included in this list)
- 7) User hits “Submit” (or happens right after a Summary is Chosen?)
- 8) iTuff Parser starts
 - a. iTuff tokens used for Lot / Part information:

i. 6_lotid_	Lot ID
ii. 6_prgnm_	Test Program Name
iii. 6_packg_	Package
iv. 6_prdct_	Product
v. 6_testertype_	Tester Type
vi. 6_comnt_socfile_	Socket File
vii. 5_lcode_	Location Code / Operation
viii. 4_smrynam_	Summary Name
ix. 4_sysid_	Tester Name
x. 4_tempr_	Temperature
xi. 4_ldbid_ / _tiuid_	Tiu ID
xii. 3_lsep	Unit Separator
xiii. 3_prtnm_	Part Number
xiv. 3_socket_	Socket Number
xv. 3_curfbn_	Function Bin (i.e. 301)
xvi. 3_curibin_	Interface Bin (i.e. 3)
xvii. 3_binn_	8 digits binning (i.e. 90939356)
xviii. 2_visualid_	Visual ID
xix. 2_trlot_ / 2_sstrlot_	Sort Lot
xx. 2_trwafer_ / 2_sstrwafer_	Sort Wafer
xxi. 2_trxloc_ / 2_sstrxloc_	Sort X location
xxii. 2_tryloc_ / 2_sstryloc_	Sort Y location
 - b. iTuff header is parsed to determine the “key” for the [pinfinder_config] table:
 - i. [Package_ID_Search_Text] Ex: 6_packg_~~~

- ii. [Test_Program_Search_Text] Ex: 6_prgnm_~~~
 - iii. [Interface_Unit_ID_Search_Text] Ex: 4_ldbid_~~~, 3_tuid_~~~
 - iv. [Tester] Ex: 6_testertype_~~~
- c. [pinfinder_config] is queried to match the above conditions and returns the row containing the above fields as well as:
 - i. [Interface Unit Design ID]
 - ii. [Manufactured Unit Design ID]
- d. If no match is found, then the product isn't configured properly and Error is returned
- e. iTuff file is parsed and output (failing channels) stored on a JSON
 - i. Call Trace to find out which failing test name prefix matches the user's bin
 - ii. If no match from Trace, determine which key words in the **2_tname** field corresponds to the user's bin(s)
 - 1. Read "BinTokens" JSON text file to look up the keys words for each bin
 - a. "8": "HWALARM,!DFF,cont,CONT,vcc,Cont,VCC,Vcc,shorts",
 - b. "9": "leak,LEAK,Leak,LKG,lkg,dc_lk",
 - c. "10":
"SHOPS,EDM,edm,open,Open,OPEN,short,Short,SHORT,shops,op,cont,upper,lower,dc_shrts,OS_!VCC,!Time,OS,AZALIA",
 - d. "11":
"MIO,SIO,UPI,MIO,DDR,VIX,VOX,VIL,VIH,VOL,VOH,Vix,Vox,Rodt,vix,vox,vil,vih,Vix,Vox,Rodt,AGTL_Ron,CMOS_OD_Via,VIA,!EDC,IOPMU,PCHIODC,PCHRTCG3,!TTR",
 - e. "12": "bf,BF,psmi,PSMI,COREFT,First,!hmoo,rcomp",
 - f. "13":
"SIO,UPI,MIO,DDR,VIX,VOX,VIL,VIH,VOL,VOH,Vix,Vox,Rodt,vix,vox,vol,voh,Vix,Vox,Rodt,AGTL_Ron,CMOS_OD_Via,VIA,via,!EDC,pll,rup,rdn,IOPMU,!LEAK,!SCRBRD,!BSCANLKG",
 - g. "15":
"!Negative,!ttr,SHOPS,Shops,open,opens,Open,OPEN,short,Short,SHORT,shops,op,cont,upper,lower,EDM,edm,dc_shrts,OS_!VCC,!Time,!tt_!testtime,OS,!DTS,qpiDC,shodc,SHODCXH,extest,!e",
 - h. "16": "**IO_TIMING,DRAM**,UBUMP,AC,ACIO,acio,io,IOLB,PEG,peg,Peg,Dmi,dmi,RTERM,!tion,!EDM",
 - i. "19": "reset,RESET,Reset,pll,PLL",
 - j. "22": "RTERM,Rterm,VIA,Rss,Rodt",
 - k. "25": "buf,bscan,BScan,BSCAN,tog",
 - l. "27": "ssid,SSID,io,IO,FDI,FIVR,LDO",
 - m. "28": "cres,CRES,pll",
 - n. "29": "HBMIO,FBIST,SCN",
 - o. "30": "VIPR",
 - p. "32": "rcomp,RCOMP",
 - q. "34": "MDFI,**DSI**",

- r. "35": "DFX,HTD,!delay",
- s. "36": "FBIST",
- t. "37": "ttr,cont,CONT,vcc,Cont,_",
- u. "40": "vohm",
- v. "41": "scan,chain",
- w. "42": "scan,chain,atpg,ATPG,SCN",
- x. "43": "io,IO,sata,!tio,TRIM",
- y. "44": "!tpg,sbft,SBFT,st_full,fcdragon",
- z. "46":
"SIO,UPI,buffer_toggle,xor,XOR,rxdetCMTSocketFile,ntl,viadelam,rxdet",
- aa. "48": "SIO,IBIST,ntl,viadelam,rxdet,!SCRBRD",
- bb. "50": "CPGC,cpgc",
- cc. "51": "ODT,odt",
- dd. "52": "UNCOREFT,iobist",
- ee. "54": "pcie,PCI,PIO,pio,IO,!PMU,!PSIO",
- ff. "60": "ARR,CORE,PBIST,pbist",
- gg. "62": "VOLTAGE",
- hh. "64": "THERMAL,thermal",
- ii. "67": "visualid,visid,VISUALID",
- jj. "68": "MIO,DDR,IOFS",
- kk. "74": "RTERM",
- ll. "75": "RTERM",
- mm. "79": "RTERM",
- nn. "94": "HWALARM",
- oo. "97": "HANDLER,CHUCK",
- pp. "98": "HWALARM,leak,LEAK,Leak,LKG,lkg,_",
- qq. "99": "HWALARM,!diva",
- rr. "all": "failbump,faildata"

iii. Supported iTUFF formats:

1. Format 1

- a. 2_tname_~~~~
- b. 2_category_fc
- c. 2_composite_<channel1>_<measurement1>
- d. 2_composite_<channel2>_<measurement2>
- e. ...
- f. 2_composite_<channelN>_<measurementN>
- g. 2_msunit_<units>

2. Format 2

- a. 2_tname_~~~~
- b. Support both pin_name|measurement or pin_name|pin_name
 - i. 2_strgval_<N
failures>|<Pin1>|<measurement1>|<Pin2>|<
measurement2>|...|<PinN>|< measurement3>|

- ii. 2_strgval_<N failures>|<Pin1>|<Pin2>|<Pin3>|<Pin4>|...|<PinN>|
 - c. 2_comnt_failpins_{<channel1>,<channel2>,...,<channelN>}
- 3. **Format 3**
 - a. 0/2_tname_~~~~
 - b. 0/2_mrslt_<measurement>
 - c. 0/2_msunit_<units>
 - d. 0/2_comnt_faildata_{<channel1>}
 - e.
- 4. **Format 4 (Shops test for CLASS and BURN-IN)**
 - a. 2_tname_~~~~
 - b. 2_pttrn_DomainName:FailingPattern:FailingPatternList
 - c. ...
 - d. 2_faildata_{<channel1>,<channel2>,...,<channelN>}
 - i. Prefix of "9" is put in front of the "DUT-mapped" output Channel# for each input pin that fails to indicate that this is not the actual channel that failed (post processing)
 - e. 2_comnt_failpins_{<Pin1>,<Pin2>,...,<PinN>}
 - f. **Burn-in test**
 - i. For the HBI tester, it will not use Tester Channel Number as the decoding criteria just like other tester, instead it will use the Socket Channel Number i.e. 15099 – "15" is the HBI socket number and "099" is the channel number. The last three digit is always going to be the channel number
 - ii. In HBI tester, there will be 16 sockets.
 - iii. The current supported HBI bins are only for I/O connectivity with SHOPS and RESET failure which are for bin 10, 15 and 19
- 5. **Format 5 (VIX/VOX Search)**
 - a. 2_tname_~~~~
 - b. 2_category_fc<num>
 - c. 2_pttrn_DomainName:FailingPattern:FailingPatternList
 - d. 2_comnt_A!_composite_9<channel1>_<measurement1>
 - i. Prefix of "9" is put in front of the "DUT-mapped" output Channel# for each input pin that fails to indicate that this is not the actual channel that failed (post processing)
 - e. 2_comnt_A!_composite_9<channel2>_<measurement2>
 - f. ...
 - g. 2_comnt_A!_composite_9<channelN>_<measurementN>
 - h. 2_msunit_<units>
 - i. 2_comnt_failpins_{<Pin1>,<Pin2>,...,<PinN>}

6. **Format 6** PCAT (Eltuff) – different iTUFF file – only parse this file if specifically requested and report whatever is found
 - a. Old PCAT format
 - i. 2_tname_**PF_PCAT_FailData** (special token)
 - ii. 2_strgval_<channel1>|<channel2>|...|<channelN>
 - b. New PCAT Format (Eltuff) – use this format if not “PF_PCAT_FailData” is found in the PCAT file (backward compatible)
 - i. 2_tname_VOL_FailPins
 - ii. 2_strgval_<Pin1>|<Pin2>|...|<PinN>
 1. Use the Socket file to translate to Channel?
7. **Format 7** OPIO:
 - a. 0_tname_~~~~_faildata
 - b. 0_strgval_<Pin1>,<Pin2>,...,<PinN>
8. **Format 8** RCOMP:
 - a. 0_tname_~~~~_FailPins
 - b. 0_strgval_<Pin1>|<Pin2>|...|<PinN>
9. **Format 9** No touch or Power Rail (HDMX/T) (1):
 - a. 0_tname_~~~~_[af]c\d
 - b. 0_strgval_<N failures>|<Pin1>|<measurement1>|<Pin2>|<measurement2>|...|<PinN>|<measurement3>|
 - c. **No touch pin decoding note:**
 - i. All the no touch Pin will have a dummy “Tester Channel Number” that is illustrated as negative number value in the [Pinfinder database](#). However, these “Tester Channel Number” information will not be displayed in the Pinfinder ITuff prober.
 - ii. The “Name” will be use as the matching criteria in no touch pin instead of the “Tester Channel Number” in the normal touched pin. These “Name” or the other name for it is “Net Name” should match the “Signal Name” data from our [Pinfinder Supported Product](#)
 - iii. You can select the product database base on clicking on the related product configuration in the [Pinfinder Supported Product](#) webpage
 - d. **Power rail decoding note (HDMX/HDMT) (1):**
 - i. All the power rail Pin will have “Tester Channel Number” that is illustrated as the number that has value of 0 to 999 and 11000 to 11999 in the [Pinfinder database](#). This “Tester Channel Number” information will be displayed in the Pinfinder ITuff prober.
 - ii. The “Name” will be use as the matching criteria in no touch pin instead of the “Tester Channel Number” in the normal touched pin. These “Name” or the other

name for it is “Net Name” should match the “Signal Name” data from our [Pinfinder Supported Product](#)

- iii. There’s possibility of having more than one “Tester Channel Number” for one “Net Name”. For this case, we will put on tagging for that particular row in the Pinfinder for the “Tester Channel Number”
- iv. Example: <Tester Channel Number 1>|<Tester Channel Number 2>|...|<Tester Channel Number N>
- v. There’s possibility of having more than one “Net Name” for one “Tester Channel Number”. For this case, Pinfinder will grab the “Net Name” from the iTuff and will detect all the other “Net Name” that have the same “Tester Channel Number” from the Pinfinder database. All the other “Net Name” will be displayed for the user

10. Format 10 Power Rail (HDMX/T) (2):

- a. It can follow any format as long as a clear **Test_Name** is available
- b. **Power rail decoding note (HDMX/HDMT) (2):**
 - i. The **Test_Name** from iTuff will be matched with the **Test_Name** from the Bin8/Bin99 lookup table from the Pinfinder database to get the Net_Name
 - ii. This will always be the end result (overwrite the normal decoding result) as it is a final layer of decoding provided that the **Test_Name** from the iTuff is matching the **Test_Name** from the Pinfinder database
 - iii. Link to Bin8/Bin99 lookup table: http://pinfinder.intel.com/PinFinder/avacado/lookup_table or it can be found at the bottom of Pinfinder iTuff prober page

11. Format 11 Function Test support:

- a. 2_tname_~~~~
- b.
- c. 2_pttrn_domain:<Pattern>(:Plist)
- d.
- e. 2_vcont_<Vector>
- f.
- g. 2_faildata_{<channel1>,<channel2>,...,<channelN>}

12. Format 12.A Alarm Log:

- a. 2_tname_<HWALARM_~~~>|Pin:<Pin>|Type:<Type>
- b. 2_strgval_Module:<Module>|Block:<Block>|Connector:<Connector>|BusPort:<BusPort>|Time:0

13. Format 12.B Alarm Log:

- a. 2_tname_<HWALARM_~~~>|Pin:<Pin>|Type:<Type>

- b. **2_strgval_Module**:<Module>|**Channel**:<Block>|**Connector**:<Connector>|**BusPort**:<BusPort>|Time:0
- c. Time format can be more than 50 characters
- d. **Alarm log decoding note (HDMX/HDMT)**:
 - i. The **Test_Name** and the **Pin** from iTuff will be matched with the **Test_Name** and the **Substrate Net Name** respectively from the Bin8/Bin99 lookup table in the Pinfinder database to get the Net_Name
 - ii. This will always be the end result (overwrite the normal decoding result) as it is a final layer of decoding provided that the **Test_Name** and the **Pin** from the iTuff is matching the **Test_Name** and the **Substrate Net Name** from the Pinfinder database

14. **Format 13** EMIB format (1):

- a. [0|2]_tname_<test_name>|<pin>_failbump
- b. [0|2]_strgval_<failpat1>|<failpat2>|...|<failpatN>
- c. For SSID die decoding:
 - i. [0|2]_tname_<test_name>|<pin>_DIEXX_failbump
 - ii. **XX** can be 00, 01, 10, 11

15. **Format 14** EMIB format (2): (might have limitation on the tname length)

- a. [0|2]_tname_<test_name>|<pin1>|<pin2>|...|<pinN>_failbump
- b. [0|2]_strgval_<failpat1>|<failpat2>|...|<failpatN>
- c. For SSID die decoding:
 - i. [0|2]_tname_<test_name>|<pin>_DIEXX_failbump
 - ii. **XX** can be 00, 01, 10, 11

16. EMIB format (3): (to overcome the limitation on format (2)) – applicable in next patch

- a. [0|2]_tname_<test_name>|<pin1>_failbump
- b. [0|2]_strgval_<pin2>|<pin3>|...|<pinN>_failbump
- c. For SSID die decoding:
 - i. [0|2]_tname_<test_name>|<pin>_DIEXX_failbump
 - ii. **XX** can be 00, 01, 10, 11

- f. Failing Channels from JSON are translated to Failing Signal Names by database lookup:
 - i. Look up Tester channel in [tiu_substrate] table (this table needs to be loaded by the Scheduled Task “PinFinderMidasLoader” on sc11ssql301)
 - ii. Match the row that contains:

1. [Package_ID_Search_Text]	from [pinfinder_config]
2. [Test_Program_Search_Text]	from [pinfinder_config]
3. [Interface_Unit_ID_Search_Text]	from [pinfinder_config]
4. [Interface Unit Design ID]	from [pinfinder_config]
5. [Manufactured Unit Design ID]	from [pinfinder_config]
6. [Tester_Channel_Num]	iTUFF failing channel

- 7. [Socket_ID] 3_socket_~~~
- 8. [Valid_Flag] Y (N are old invalid rows...)

iii. Output row contains:

- 1. [Connector_Name] / [Connector_ID] Ex: AB12
- 2. [Manufactured_Unit_Signal_Name] Signal Name
- 3. [Connector_Location_Flag] Top or Bottom Connector
- 4. [Tester_Channel_Pin_Name] Tester Channel to debug Tester

9) SIID binning display support

- a. Will be supporting all the product that contain all of below conditions:
 - i. Contain die information within the iTuff which satisfies the Pinfinder spec format
 - ii. Supportable product is the product that have up to 4 dies
 - iii. Supportable product is the product that have more than 1 die

10) 8 Digit binning support

- a. Will be supporting all the product that contain all of below conditions:
 - i. Contain die 8 digit binning information in the iTuff (mainly server product)
 - ii. Supportable Ituff format for 8 digit binning is
 - 1. 3_binn_*****
 - 2. Where the * is the bin digits

11) Report Results to User